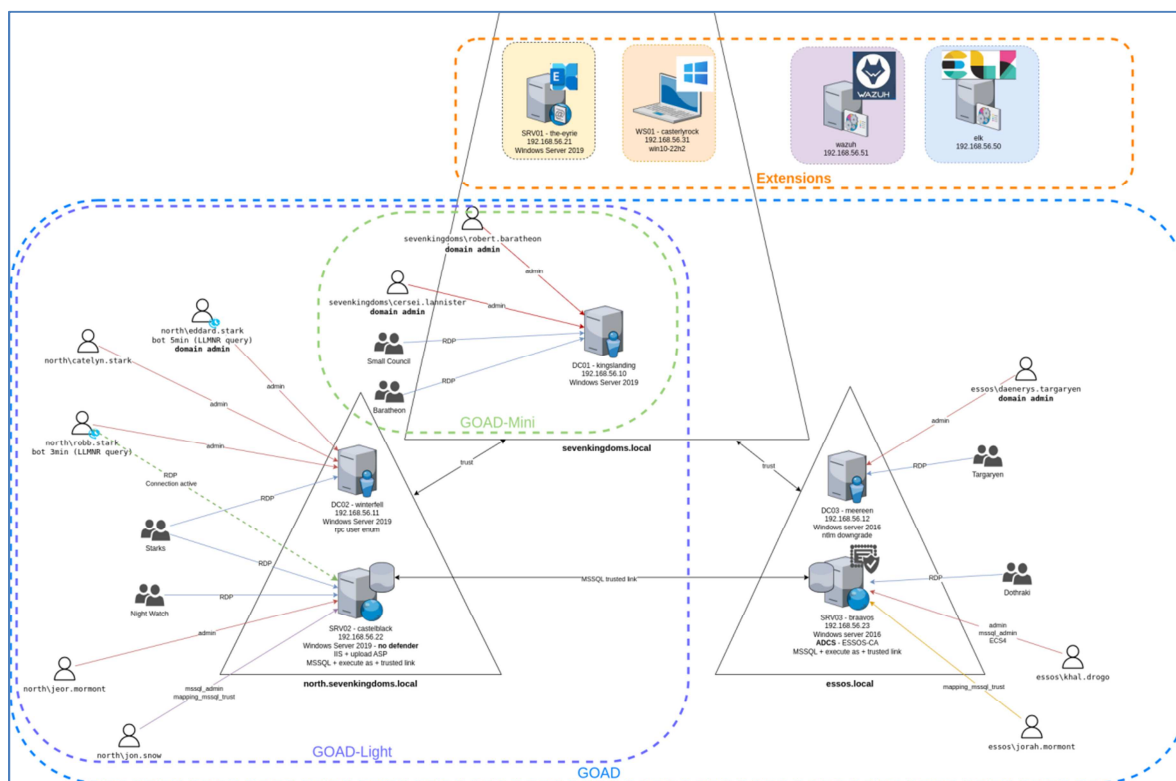


GOAD Active Directory + ELK Lab Setup from a Windows host

JJKim 12.29.2025

Preface

This write up is to document the install of [GOAD – Game of Active Directory v3](#) using **VMware Workstation Pro** on a **Windows 11 OS PC (host)**. I updated the [previous Setup](#) to install and provision ELK.



GOAD: 5 VMs, 2 forests, 3 domains (full GOAD lab) + ELK VM

This install requires that you have a **Windows (10 or 11) OS PC host** with **VMware Workstation Pro** (VMware-workstation-17.6.3.24583834.exe) installed and you have downloaded the following ISOs:

- **Ubuntu 20.04 Desktop:** ubuntu-22.04.5-desktop-amd64.iso
- **Kali Linux:** kali-linux-2025.2-installer-amd64.iso

SECTION OVERVIEW:

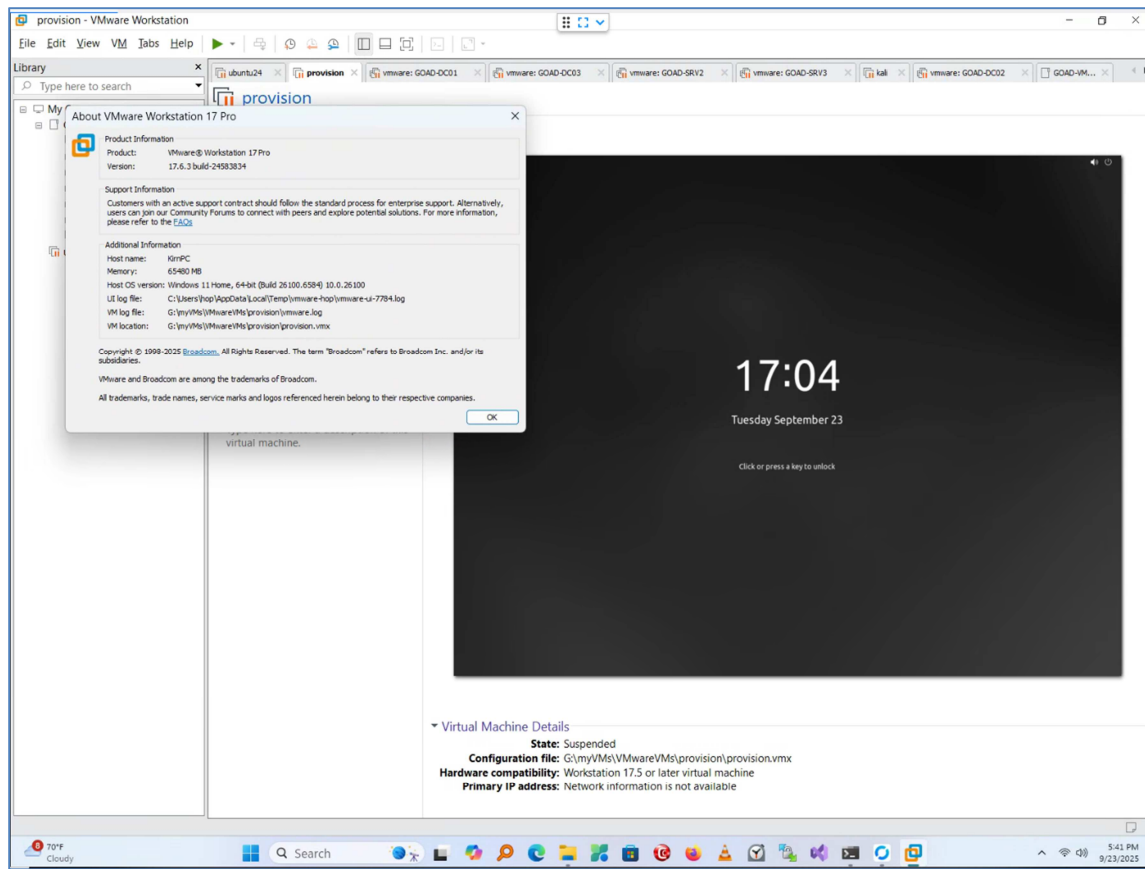
1. **Section A - Create the 5 GOAD Windows VMs and the ELK Linux VM.** The details in creating the VMs using only vagrant are covered in this section.
2. **Section B – Provisioning.** The details in creating the Ubuntu 20.04 VM used for provisioning for all 5 GOAD VMs and the ELK Linux VM using only ansible are covered in this section.
3. **Section C - Attack Techniques.** The details in creating the kali VM and running our first attack command are covered in this section.
4. **Section D - Extra Information.** This section provides helpful in answering questions about the installs not covered elsewhere.

GENERAL NOTES:

1. We will use our **Windows 11 OS host** with the installed **VMware Workstation Pro** together with an install of **GOAD** (<https://github.com/Orange-Cyberdefense/GOAD.git>) to create the first environment. In this environment you will only need to run **vagrant** to create the 5 GOAD VMs and the ELK VM. See **Section A. Create the 5 GOAD Windows VMs and ELK Linux VM** below for details.
2. The **Ubuntu 22.04 Desktop VM** you create will only be used for “**provisioning**”. You will also need to install **GOAD** (<https://github.com/Orange-Cyberdefense/GOAD.git>) on that VM. You will only need to run **ansible** in this environment for the **provisioning**. See section **B. Provisioning** below for details.

A. Create the 5 GOAD Windows VMs and ELK Linux VM for VMware Workstation Pro

1. **First step is to make sure you have VMware Workstation Pro installed:**



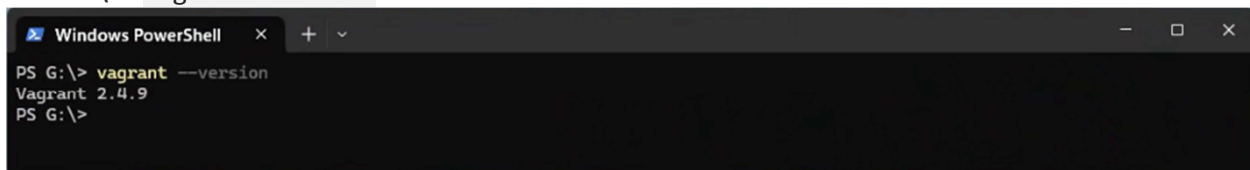
VMware Workstation Pro Install

To download and install VMware Workstation Pro 17 for Windows, go to the [Broadcom support portal](#), create an account if you don't have one, and log in and download the latest version.

2. Next you need to download and [install the “Vagrant VMware Utility”](#) for Windows.

When you complete the Vagrant installation, you can check whether the installation was successful via powershell (reminder this is on the Windows 11 host):

```
PS G:\> vagrant -version
```



Vagrant Version

-added

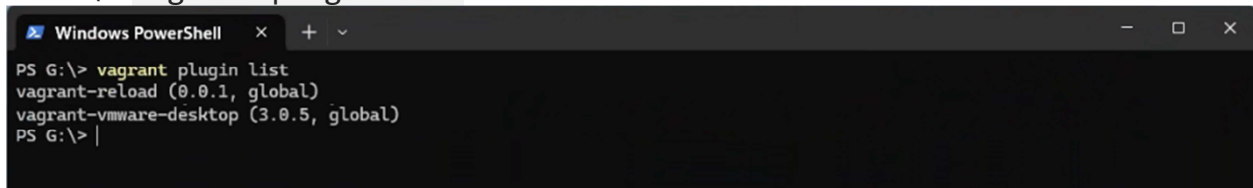
Open PowerShell as Administrator and run:

```
vagrant plugin install vagrant-vmware-desktop
vagrant plugin install vagrant-reload
vagrant plugin install winrm
vagrant plugin install winrm-fs
vagrant plugin install winrm-elevated
```

end of add

You also need two vagrant plugins:

```
PS G:\> vagrant plugin list
```



Vagrant Plugin List

3. Now you need to download and install the GOAD lab and use vagrant to create the 5 Window VMs and ELK Linux VM (Again, this part is done on the Windows 11 host directly)

Open powershell and go to the location where you want to install GOAD:

```
PS G:\> git clone https://github.com/Orange-Cyberdefense/GOAD.git
```

Start by fixing the "Vagrantfile" to be used for the install:

```
PS G:\GOAD\ad\GOAD\providers\vmware> cat .\Vagrantfile
```

```
#--- JJK 12/29/25, https://github.com/Orange-Cyberdefense/GOAD
Vagrant.configure("2") do |config|
```

```
ENV['VAGRANT_DEFAULT_PROVIDER'] = 'vmware_desktop'
```

```
config.vm.boot_timeout = 600 # this stuff is from template
config.vm.graceful_halt_timeout = 600
config.winrm.retry_limit = 30
config.winrm.retry_delay = 10
```

```
boxes = [
  {
    name: "GOAD-DC01",
    ip: "192.168.56.10",
    box: "StefanScherer/windows_2019",
    box_version: "2021.05.15",
    os: "windows",
```

```

    cpus: 2,
    mem: 3000
  },
  {
    name: "GOAD-DC02",
    ip: "192.168.56.11",
    box: "StefanScherer/windows_2019",
    box_version: "2021.05.15",
    os: "windows",
    cpus: 2,
    mem: 3000
  },
  {
    name: "GOAD-DC03",
    ip: "192.168.56.12",
    box: "StefanScherer/windows_2016",
    box_version: "2017.12.14",
    os: "windows",
    cpus: 2,
    mem: 3000
  },
  {
    name: "GOAD-SRV2",
    ip: "192.168.56.22",
    box: "StefanScherer/windows_2019",
    box_version: "2021.05.15",
    os: "windows",
    cpus: 2,
    mem: 6000
  },
  {
    name: "GOAD-SRV3",
    ip: "192.168.56.23",
    box: "StefanScherer/windows_2016",
    box_version: "2019.02.14",
    os: "windows",
    cpus: 2,
    mem: 5000
  },
  {
    name: "GOAD-ELK",
    ip: "192.168.56.50",
    box: "bendto/ubuntu-24.04",
    os: "linux",
    cpus: 2,
    mem: 4096
  },
# },
#   name: "GOAD-BUILDER", # This is if you wanted to Ubuntu server box
#   ip: "192.168.56.100",

```

```

#
#   box: "bento/ubuntu-22.04",
#   cpus: 2,
#   mem: 2048,
#   os: "linux"
}

]

boxes.each do |opts|
  config.vm.define opts[:name] do |vm|
    vm.vm.box = opts[:box]
    vm.vm.box_version = opts[:box_version]
    vm.vm.hostname = opts[:name] # This sets the VM hostname inside the OS
    # issues/49
    vm.vm.synced_folder '.', '/vagrant', disabled: true
    vm.vm.network "private_network", ip: opts[:ip]

    vm.vm.provider :vmware_desktop do |v|
      v.vmx["memsize"] = opts[:mem].to_s
      v.cpus = opts[:cpus]
      # jjk 9.18.25
      # v.force_vm_license = "workstation" #force the license for some
vagrant plugin issues
      v.vmx["ethernet0.pciSlotNumber"] = "192" # Fix warning
      v.vmx["ethernet1.pciSlotNumber"] = "224" # Fix warning
    end

    # Optional shell provisioner - JJK, from template
    if opts[:os] == "windows"
      vm.vm.guest = :windows
      vm.vm.communicator = "winrm"
      vm.vm.provision :shell, :path => "G:/GOAD/vagrant/Install-
WMF3Hotfix.ps1", privileged: false
      vm.vm.provision :shell, :path =>
"G:/GOAD/vagrant/ConfigureRemotingForAnsible.ps1", privileged: false

      # fix for vmware static ip
      vm.vm.provision :shell, :path => 'G:/GOAD/vagrant/fix_ip.ps1',
privileged: true, args: opts[:ip]
      # also enable ping response
      vm.vm.provision :shell, :path => 'G:/GOAD/vagrant/fix_ping.ps1',
privileged: true

    else
      vm.vm.communicator = "ssh"
    end

  end
end
end
end

```

end

Next you need to fix the IP addresses in the “inventory” file located at the same location:
We are now ready to create all the 5 GOAD Windows VMs using the following powershell command at the location indicated:

```
PS G:\GOAD\ad\GOAD\providers\vmware> cat inventory
```

```
[default]
; Note: ansible_host *MUST* be an IPv4 address or setting things like DNS
; servers will break.
; -----
; sevenkingdoms.local
; -----
dc01 ansible_host=192.168.10 dns_domain=dc01 dict_key=dc01
; -----
; north.sevenkingdoms.local
; -----
dc02 ansible_host=192.168.56.11 dns_domain=dc01 dict_key=dc02
srv02 ansible_host=192.168.56.22 dns_domain=dc02 dict_key=srv02
; -----
; essos.local
; -----
dc03 ansible_host=192.168.56.12 dns_domain=dc03 dict_key=dc03
srv03 ansible_host=192.168.56.23 dns_domain=dc03 dict_key=srv03
elk ansible_host=192.168.56.50 dns_domain=dc03 dict_key=dc03 ;not sure this is needed
```

```
PS G:\GOAD\ad\GOAD\providers\vmware> vagrant up
```

4. You need to check and update the IPs of all the Windows Machines:

The Vagrant script is supposed to correctly “fix the IP” on Windows VMs. However for some reason it is not always successful. I observed that although the VMs were created, they didn’t have the right IPs.

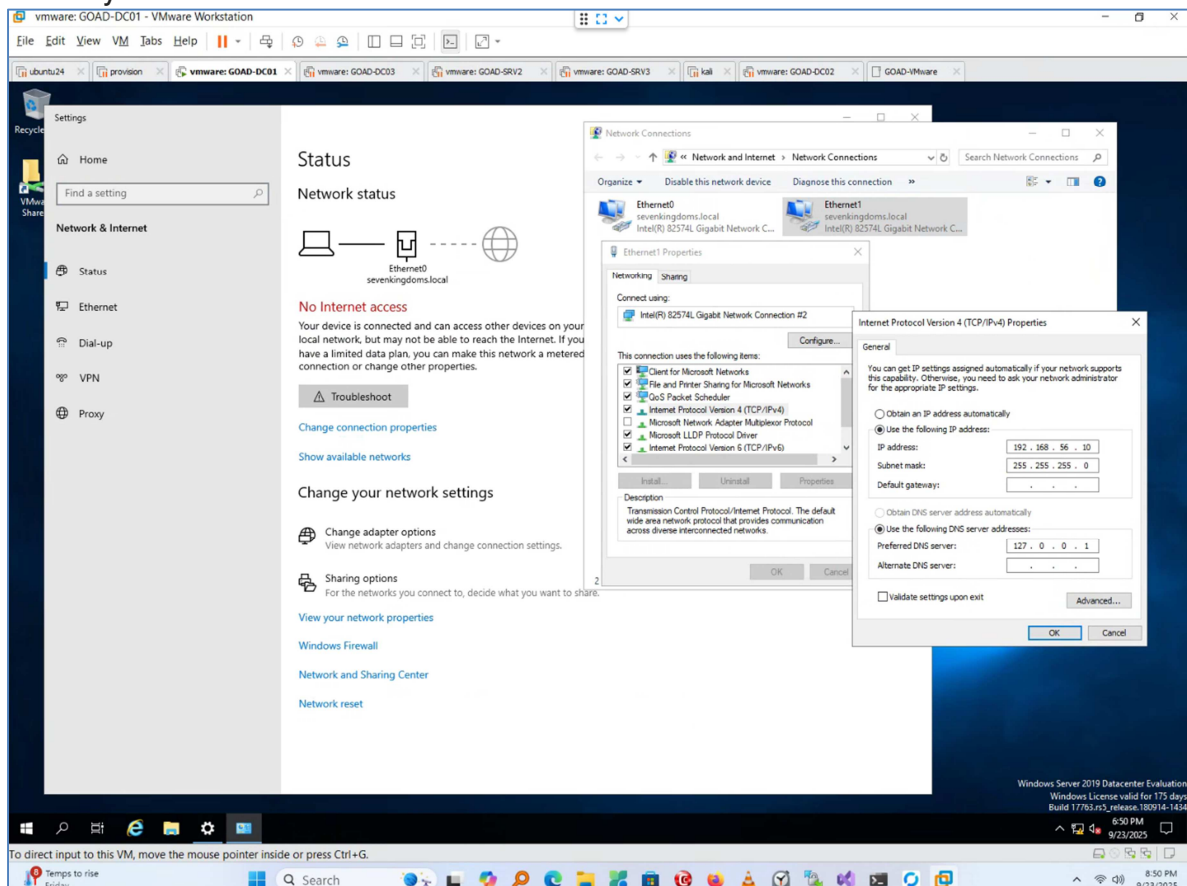
Several times had to manually set the Windows VM IPs as follows:

Server	IP	MASK
DC1	192.168.56.10	255.255.255.0
DC2	192.168.56.11	255.255.255.0
DC3	192.168.56.12	255.255.255.0
SRV02	192.168.56.22	255.255.255.0
SRV03	192.168.56.23	255.255.255.0

The GOAD-ELK VM (Linux) info should be fine:

Server	IP	MASK
ELK	192.168.56.50	255.255.255.0

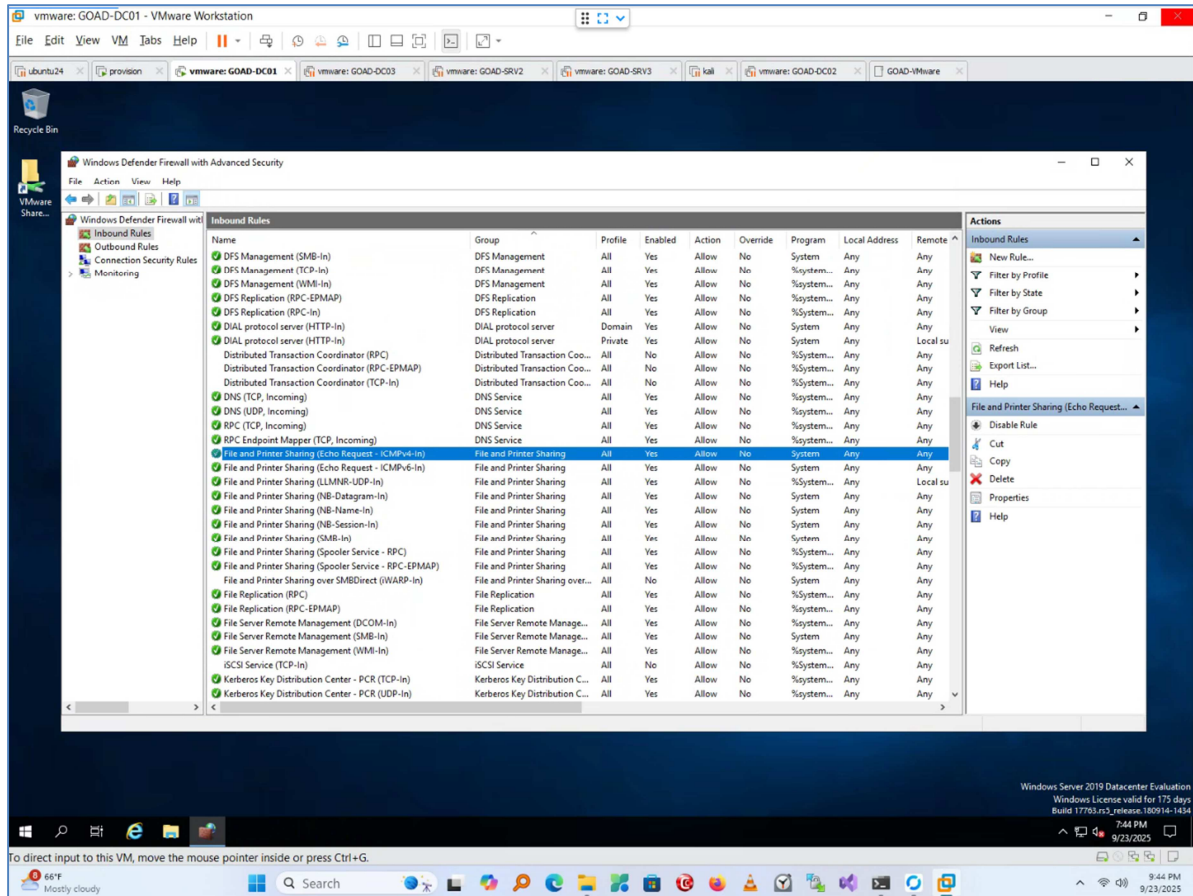
I logging into each of the 5 Windows GOAD VMs as **vagrant:vagrant** and made changes if necessary:



Example for DC01

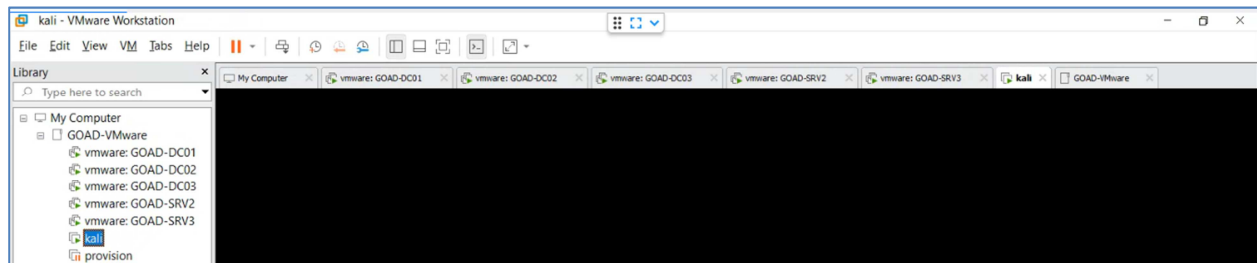
5. You also need to verify “enable ping” on all **Windows** servers:

The Vagrant script is supposed to “enable ping” response on Windows VMs. However for some reason it is not always successful. To verify “enable ping” was performed correctly, open the Windows Defender Firewall with Advanced Security console, select Inbound Rules, and enable the pre-existing rule named "File and Printer Sharing (Echo Request - ICMPv4-In)" if necessary. This needs to be done on each of the 5 GOAD Windows VMs.



Example for DC01:

Assuming all goes well with vagrant up, you should now see the 5 newly created **GOAD VMs**:



GOAD VMs

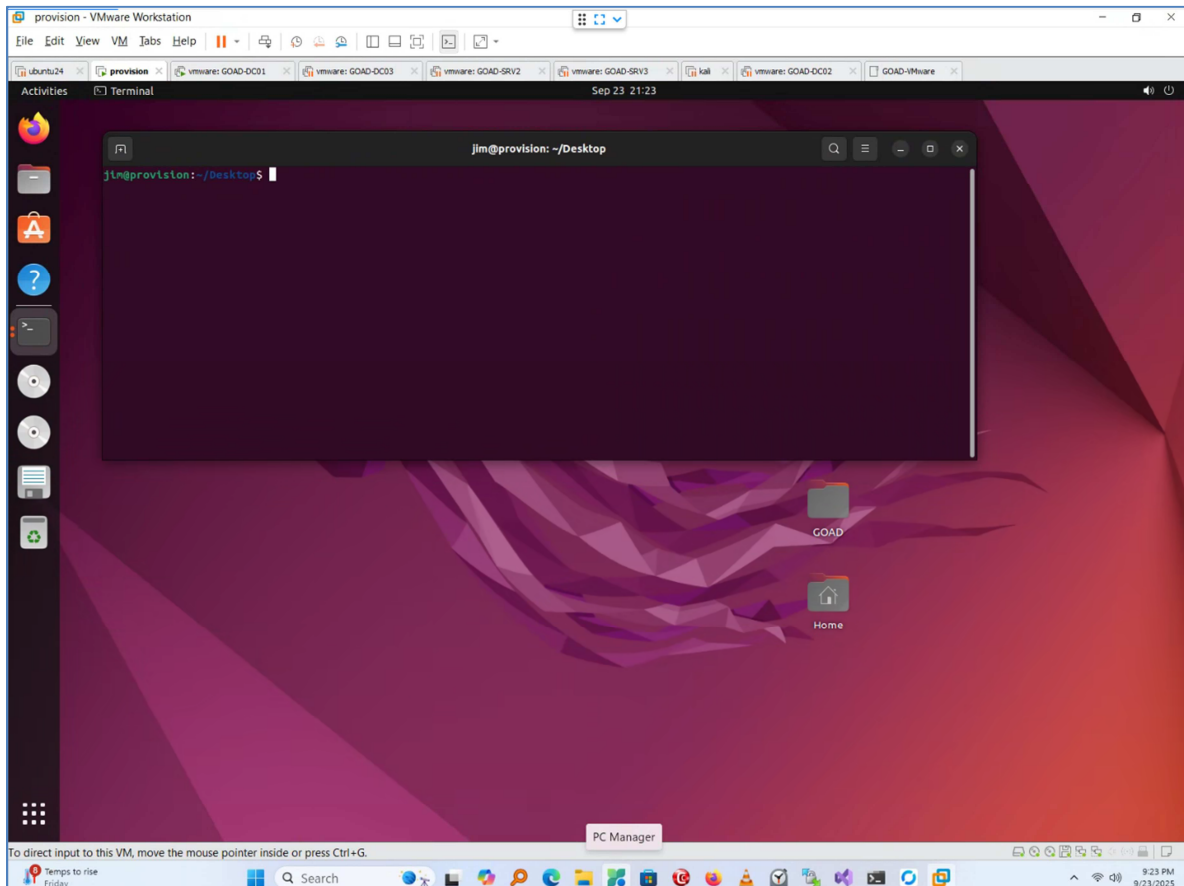
B. Provisioning (done via an Ubuntu 22.04 Desktop VM)

Great! We have created all the 5 Windows GOAD VMs and the ELK Linux VM, now we need to provision the AD environment and ELK!

Let's switch to the Ubuntu "provision" VM that I manually created from ISO:

- RAM: 4 GB
- Processors: 2

- HD: 60GB
- VM: Ubuntu 22.04 Desktop
- Network Adapter 1: NAT
- Network Adapter 2: VMnet2
 - a. Static IP 192.168.56.100



Start of Provisioning

I'm not sure you need this step, but it won't hurt and it helps with ping verifications. Update the /etc/hosts file to include the IP addresses for our GOAD VMs:

```
jim@provision:~/Desktop$ cat /etc/hosts:
```

```
127.0.0.1    localhost
127.0.1.1    provision
```

```
# The following lines are desirable for IPv6 capable hosts
::1          ip6-localhost ip6-loopback
fe00::0      ip6-localnet
ff00::0      ip6-mcastprefix
ff02::1      ip6-allnodes
ff02::2      ip6-allrouters
```

```
# GOAD IPs
192.168.56.10  dc01  DC01
192.168.56.11  dc02  DC02
192.168.56.12  dc03  DC03
192.168.56.22  srv02  SRV02
192.168.56.23  srv03  SRV03
192.168.56.50  elk    ELK
```

Next I started the provisioning part by installing all requirements before running ansible to perform the provisioning.

To install python virtual environment:

```
sudo apt update
sudo apt upgrade

sudo apt install git
git clone http://github.com/Orange-Cyberdefense/GOAD.git

cd GOAD/ansible
```

NOTE:

Ansible is currently stable with Python versions 3.8 through 3.11. To resolve any potential issues use python3.10

```
sudo apt install python3.10-venv
python3.10 -m venv .venv

source .venv/bin/activate
```

To install ansible pywinrm in the .venv:

```
python3 -m pip install --upgrade pip
python3 -m pip install ansible-core==2.12.6
python3 -m pip install pywinrm
```

The following command is outside of the GOAD directory, so if you are copy and pasting make sure to cd to GOAD. Install all the ansible-galaxy requirements:

```
ansible-galaxy install -r ansible/requirements.yml
```

Time to add the ELK Extension to the Existing Full GOAD Install

1. SSH into your Ubuntu provisioning VM (192.168.56.50)

The “provisioning” VM is already in VMware Workstation Pro assuming you followed my Full GOAD install. In VMware Workstation, click on “provision” VM and make sure it is running.

2. Install sshpass (required for ELK installation)

```
jim@provision:~/Desktop/GOAD$ sudo apt install sshpass
[sudo] password for jim:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
sshpass is already the newest version (1.09-1).
0 upgraded, 0 newly installed, 0 to remove and 28 not upgraded.
```

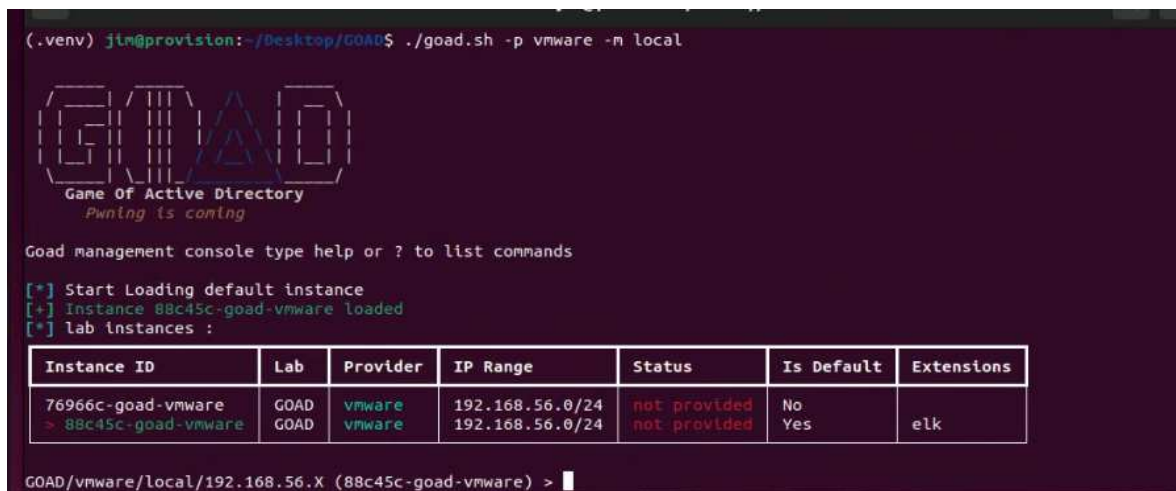
3. Navigate to GOAD directory and run the interactive shell

```
# First activate the virtual environment
jim@provision:~/Desktop/GOAD$ cd ansible
jim@provision:~/Desktop/GOAD/ansible$ source .venv/bin/activate
(.venv) jim@provision:~/Desktop/GOAD/ansible$

# Run the interactive shell
(.venv) jim@provision:~/Desktop/GOAD$ ./goad.sh -p vmware -m local

GOAD/vmware/local/192.168.56.X > provision_extension elk
```

```
(.venv) jim@provision:~/Desktop/GOAD$ ./goad.sh -p vmware -m local
```



GOAD
Game Of Active Directory
Punting is coming

Goad management console type help or ? to list commands

```
[*] Start Loading default instance
[+] Instance 88c45c-goad-vmware loaded
[*] Lab instances :
```

Instance ID	Lab	Provider	IP Range	Status	Is Default	Extensions
76966c-goad-vmware	GOAD	vmware	192.168.56.0/24	not provided	No	
> 88c45c-goad-vmware	GOAD	vmware	192.168.56.0/24	not provided	Yes	elk

```
GOAD/vmware/local/192.168.56.X (88c45c-goad-vmware) > |
```

4. Update the ELK inventory file:

```
jim@provision:~/Desktop/GOAD/workspace/88c45c-goad-vmware$ gedit elk_inventory
```

```
[elk_server]
elk ansible_host=192.168.56.50 ansible_user=vagrant ansible_ssh_pass=vagrant
ansible_become_pass=vagrant ansible_port=22 ansible_connection=ssh
```

```
[elk_server:vars]
ansible_connection=ssh
ansible_port=22
ansible_shell_type=sh
ansible_python_interpreter=/usr/bin/python3
```

```
[elk_log]
dc01 ansible_host=192.168.56.10
dc02 ansible_host=192.168.56.11
dc03 ansible_host=192.168.56.12
srv02 ansible_host=192.168.56.22
srv03 ansible_host=192.168.56.23
```

```
[elk_log:children]
```

5. Verify the ELK VM networking

```
# Check if ELK VM is reachable
# ping 192.168.56.50 # or whatever IP is in the inventory
```

```
jim@provision:~/Desktop/GOAD/workspace/88c45c-goad-vmware$ ping 192.168.56.50
PING 192.168.56.50 (192.168.56.50) 56(84) bytes of data:
64 bytes from 192.168.56.50: icmp_seq=1 ttl=64 time=0.725 ms
64 bytes from 192.168.56.50: icmp_seq=2 ttl=64 time=0.311 ms
64 bytes from 192.168.56.50: icmp_seq=3 ttl=64 time=0.322 ms
^C
--- 192.168.56.50 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2068ms
rtt min/avg/max/mdev = 0.311/0.452/0.725/0.192 ms
```

```
# Test SSH access from the "provision" VM
# ssh vagrant@192.168.56.50 (password: vagrant)
```

```
jim@provision:~/Desktop/GOAD/workspace/88c45c-goad-vmware$ ssh vagrant@192.168.56.50
vagrant@192.168.56.50's password:
Welcome to Ubuntu 24.04.2 LTS (GNU/Linux 6.8.0-64-generic x86_64)
```

```
* Documentation: https://help.ubuntu.com
* Management:   https://landscape.canonical.com
* Support:      https://ubuntu.com/pro
```

```
System information as of Mon Dec 29 09:13:06 PM UTC 2025
```

System load:	0.15	Processes:	211
Usage of /:	28.7% of 30.34GB	Users logged in:	1
Memory usage:	84%	IPv4 address for eth0:	192.168.80.133
Swap usage:	10%		

This system is built by the Bento project by Chef Software
More information can be found at <https://github.com/chef/bento>

Use of this system is acceptance of the OS vendor EULA and License Agreements.
Last login: Fri Dec 19 02:29:36 2025 from 192.168.56.100
vagrant@GOAD-ELK:~\$
vagrant@GOAD-ELK:~\$ exit
logout
Connection to 192.168.56.50 closed.

jim@provision:~/Desktop/GOAD/workspace/88c45c-goad-vmware\$

6. Test ansible on the “provision” VM can reach the ELK VM:

```
(.venv) jim@provision:~/Desktop/GOAD$ ansible -i  
/home/jim/Desktop/GOAD/workspace/88c45c-goad-vmware/elk_inventory elk_server -m ping  
elk | SUCCESS => {  
  "changed": false,  
  "ping": "pong"  
}
```

7. In case you hit the Python/Ansible `apt_key` module bug.

The error 'CustomHTTPSConnection' object has no attribute 'cert_file' is a known issue with older versions of Python's urllib3 or Ansible's apt_key module.

Solution Step 1: SSH into the ELK VM and manually add the key (quickest):

```
ssh vagrant@192.168.56.50  
# Password: vagrant  
  
# Once inside the ELK VM:  
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo gpg --dearmor -o  
/usr/share/keyrings/elasticsearch-keyring.gpg  
  
# Verify it worked  
ls -la /usr/share/keyrings/elasticsearch-keyring.gpg  
  
# Exit  
exit
```

Solution Step 2: Edit the ELK role tasks file (back on “provision” VM):

```
nano /home/jim/Desktop/GOAD/extensions/elk/ansible/roles/elk/tasks/main.yml
```

Find the task that looks like this:

```
- name: Add Elasticsearch apt key.  
  apt_key:  
    url: https://artifacts.elastic.co/GPG-KEY-elasticsearch  
    state: present
```

Replace it with:

```
- name: Add Elasticsearch apt key.  
  ansible.builtin.shell: |  
    wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | gpg --dearmor -o  
/usr/share/keyrings/elasticsearch-keyring.gpg  
  args:  
    creates: /usr/share/keyrings/elasticsearch-keyring.gpg  
  become: yes
```

Also, you'll need to update the repository addition task. Look for something like:

```
- name: Add Elasticsearch repository.  
  apt_repository:  
    repo: "deb https://artifacts.elastic.co/packages/7.x/apt stable main"  
    state: present
```

Replace it with:

```
- name: Add Elasticsearch repository.  
  ansible.builtin.shell: |  
    echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg]  
https://artifacts.elastic.co/packages/7.x/apt stable main" | tee  
/etc/apt/sources.list.d/elastic-7.x.list  
  args:  
    creates: /etc/apt/sources.list.d/elastic-7.x.list  
  become: yes
```

Save the file!

8. Then run provision again:

I previously ran it – here is how the rerun looks:


```
jlm@provision: ~/Desktop/GOAD

GOAD/vmware/local/192.168.56.X (88c45c-goad-vmware) > provision_extension elk
[*] Lab inventory : /home/jlm/Desktop/GOAD/ad/GOAD/data/inventory file found
[*] Provider inventory : /home/jlm/Desktop/GOAD/workspace/88c45c-goad-vmware/inventory file found
[*] Global inventory : /home/jlm/Desktop/GOAD/globalsettings.ini file found
[*] Run playbook : install.yml with inventory file(s) : /home/jlm/Desktop/GOAD/ad/GOAD/data/inventory,
/home/jlm/Desktop/GOAD/workspace/88c45c-goad-vmware/inventory, /home/jlm/Desktop/GOAD/workspace/88c45c-goad-vmware/elk_inventory,
/home/jlm/Desktop/GOAD/globalsettings.ini
[*] CWD: /extensions/elk/ansible
[*] Running command : ansible-playbook -i /home/jlm/Desktop/GOAD/ad/GOAD/data/inventory -l
/home/jlm/Desktop/GOAD/workspace/88c45c-goad-vmware/inventory -l /home/jlm/Desktop/GOAD/workspace/88c45c-goad-vmware/elk_inventory
-l /home/jlm/Desktop/GOAD/globalsettings.ini install.yml

PLAY [Install ELK] *****

TASK [Gathering Facts] *****
ok: [elk]

TASK [elk : Update cache] *****
ok: [elk]

TASK [elk : Add required dependencies.] *****
ok: [elk]

TASK [elk : Add Elasticsearch apt key.] *****
ok: [elk]

TASK [elk : Add Elasticsearch repository.] *****
ok: [elk]

TASK [elk : Install logstash] *****
ok: [elk]

TASK [elk : Install java] *****
ok: [elk]
```

And when it completes it looks like:

```
Terminal Dec 29 16:22
jlm@provision: ~/Desktop/GOAD

ok: [dc01]
ok: [dc02]

TASK [logs_windows : Reboot before launch setup] *****
skipping: [dc01]
skipping: [dc02]
skipping: [dc03]
skipping: [srv02]
skipping: [srv03]

TASK [logs_windows : Run winlogbeat setup] *****
skipping: [dc01]
skipping: [dc02]
skipping: [dc03]
skipping: [srv02]
skipping: [srv03]

TASK [logs_windows : check winlogbeat service] *****
skipping: [dc01]
skipping: [dc02]
skipping: [dc03]
skipping: [srv02]
skipping: [srv03]

PLAY RECAP *****
dc01 : ok=12 changed=1 unreachable=0 failed=0 skipped=11 rescued=0 ignored=0
dc02 : ok=12 changed=1 unreachable=0 failed=0 skipped=11 rescued=0 ignored=0
dc03 : ok=12 changed=1 unreachable=0 failed=0 skipped=11 rescued=0 ignored=0
elk : ok=18 changed=0 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
srv02 : ok=12 changed=1 unreachable=0 failed=0 skipped=11 rescued=0 ignored=0
srv03 : ok=12 changed=1 unreachable=0 failed=0 skipped=11 rescued=0 ignored=0

[*] Provision extension done in 18:00:54

GOAD/vmware/local/192.168.56.X (88c45c-goad-vmware) > |
```

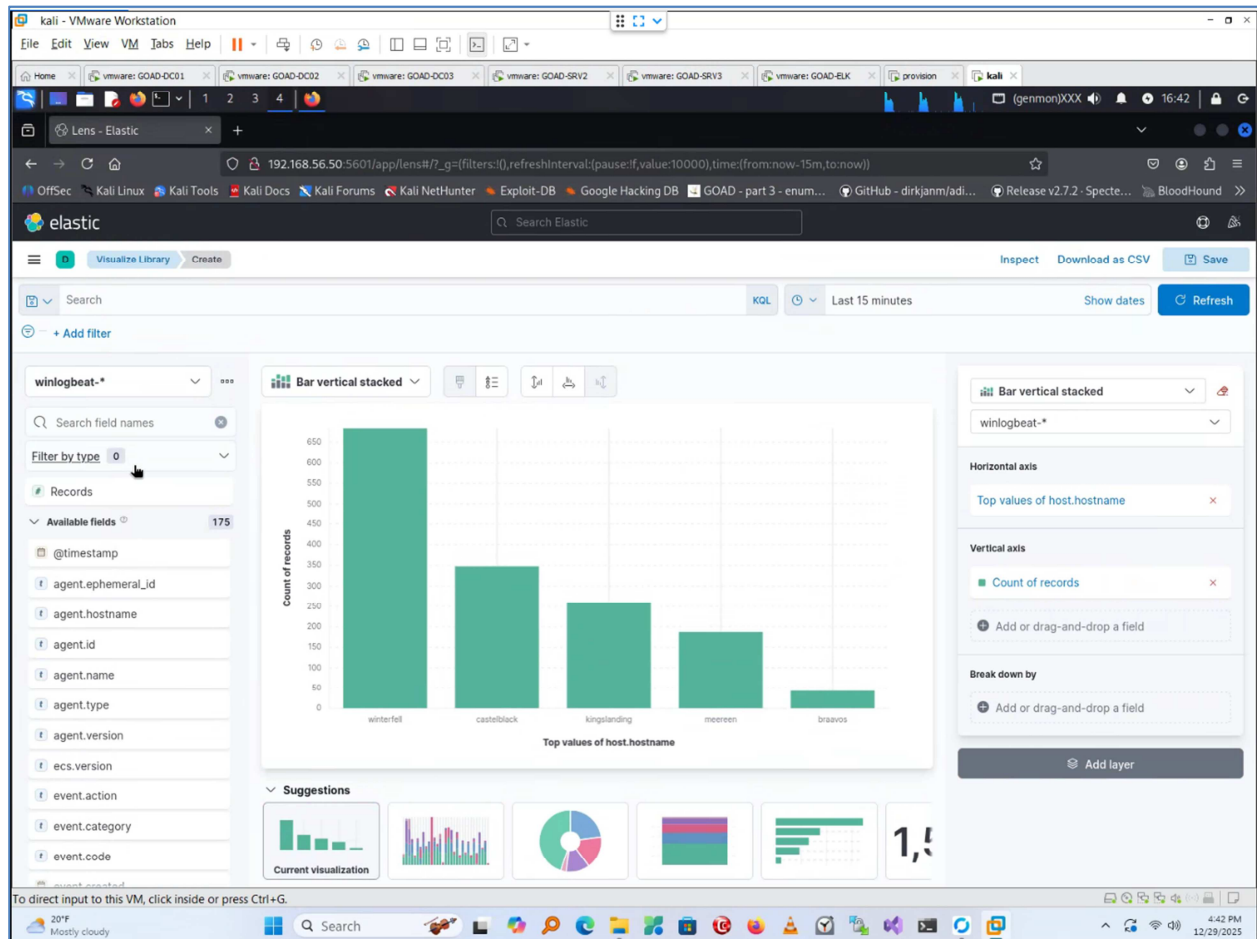
SUCCESS!

9. Access ELK via a web interface

Once the ELK installation is completed, you can access ELK at:

- <http://192.168.56.50:5601>

I accessed the web page on my **kali VM** and after some simple ELK index configuration steps, here is what I saw:



This is showing the records I am receiving from all the servers (Winter, Castelblack, Meereen, Kingslanding and Braavos). The records are updating at the rate set in ELK.

Therefore: all is working. Now I need to learn more about how to use ELK to track other security based record reporting.